

Agentic AI & Fundamentals

Week 2 — Turing x UCSB x ACM

Irene Fan & Agnibha Sarkar | March 6, 2026

irenefanyiran@gmail.com · agnibha.sarkar@turing.com

Schedule

Week	Topic
1	Course Kickoff + Problem Scoping
2	Agentic AI & Fundamentals
3	LangGraph & RAG
4	HITL Systems
5	Industry Realities of Building AI Products
6	Architecture Design & Eng Design Doc
7	Architecture Presentations + Critique
8	Coding Copilots & Dev Cycle Integration
9	End-to-End Implementation & Evaluation
10	Demo Week

Today

- 1. What is Agentic AI?
 - 2. Degrees of Autonomy
 - 3. Why Go Agentic?
 - 4. Task Decomposition
 - 5. The Four Design Patterns
 - 6. Deep Dive: Reflection
 - 7. Evaluations
 - 8. Hands-On Demo
-

How do *you* solve a hard problem?

Think about writing a cover letter for a job you really want.

Do you write it in **one shot** and hit send?

How do *you* solve a hard problem?

More likely:

1. Research the company
2. Outline key points
3. Write a draft
4. Re-read, fix the tone
5. Ask someone to review
6. Revise and send

We iterate. We use tools. We get feedback. We revise.

The One-Shot Problem

Most LLM applications today:

```
User prompt → LLM → Final answer
```

One chance. No tools. No feedback. No revision.

Works for simple questions. Falls apart for complex tasks.

The Agentic Approach

```
Prompt → LLM (plan) → Tool (search) → LLM (draft) → LLM (critique) →  
Output
```

Break tasks into steps with **loops**, **tools**, and **checkpoints**.

What makes an AI system "agentic"?

An AI agent is an LLM-powered system that can:

- **Plan** — decide what steps to take
 - **Use tools** — call APIs, databases, code, web search
 - **Remember** — carry context across steps
 - **Reflect** — check its own work and improve
-

CBRE: One-Shot vs. Agentic

One-shot:

Give the LLM a transcript. Ask it to classify, assign severity, and route — all in one prompt.

Agentic:

1. **Extract** entities from the transcript
 2. **Look up** similar past problems in a database
 3. **Classify** using extracted info + retrieved context
 4. **Check** confidence — does it need human review?
 5. **Route** to the right team
-

Degrees of Autonomy

"Agentic" isn't binary — it's a **spectrum**.

Low Autonomy

- All steps **predetermined** by the developer
- All tool use is **hard-coded**
- AI's only freedom is in text generation

Fixed pipeline — no branching, no decisions.

Semi-Autonomous

- Agent can make **some decisions** — which tools, in what order
 - All tools are **predefined**
 - Agent **chooses** when and whether to use them
-

Highly Autonomous

- Agent makes **many decisions** independently
 - Can create **new tools** on the fly
 - Plans and adapts without being told how
-

Where should the CBRE system be?

Discussion

Why Go Agentic?

1. Much Better Performance

HumanEval coding benchmark:

Approach	Accuracy
----------	----------

Approach	Accuracy
GPT-3.5 one-shot	~48%
GPT-4 one-shot	~67%
GPT-3.5 + agentic patterns	>90%

A cheaper model with a good workflow **beats** a stronger model used naively.

2. Faster Through Parallelism

An agentic system can:

- Process **50 transcripts** simultaneously
- Query **multiple databases** in parallel
- Run classification and severity assessment **at the same time**

Hundreds of calls triaged in the time a human handles one.

3. Modularity

An agentic architecture is like LEGO:

- Swap **gpt-5-mini** for **gpt-5** in one step — nothing else changes
- Replace ChromaDB with Pinecone later
- Add a weather API for HVAC correlations — no redesign needed

What tasks suit agentic AI?

Easier	Harder
Clear, step-by-step process	Steps not known ahead of time
Standard procedures to follow	Plan/solve as you go
Text-only assets	Multimodal (sound, vision)

CBRE falls in the middle. Steps are somewhat predictable, but decisions within each step depend on the transcript.

Task Decomposition

The most important skill in building agentic systems:

Break complex tasks into smaller, solvable steps.

For each step, ask:

Is this an LLM, a tool, or just code?

CBRE — Decomposed

Step	Who Does It
Extract problem type, location, severity cues	LLM
Look up matching problem codes	Tool (vector search)
Assign severity level	LLM
Check confidence score	Code
Route or flag for human review	Code

Group Exercise

Scenario: A tenant calls and says there's a strong gas smell on the 3rd floor.

Walk through the steps your system should take.

For each step: **LLM**, **tool**, or **code**?

3 minutes — pairs or small groups

Not All Workflows Are Predictable

IT helpdesk agent:

- 1. Read the issue description → **LLM**
- 2. Check knowledge base for match → **Tool**
- 3. If match → provide fix → **LLM**
- 4. If no match → ask clarifying question → **LLM** → loop back
- 5. If fix didn't work → escalate + create ticket → **Tool**

Which steps are needed **depends on what happens at each step.**

The Four Design Patterns

1. Reflection

The agent **reviews its own output** and improves it.

Like re-reading your draft before hitting send.

2. Tool Use

The LLM **calls external tools** — APIs, databases, code execution.

How agents interact with the world beyond text.

3. Planning

The agent **decides what to do next** based on what it knows so far.

Instead of hard-coding the step sequence.

4. Multi-Agent Collaboration

Multiple specialized agents work together.

One extracts, one classifies, one validates.

Design Patterns in Our Project

Pattern	Where It Shows Up	When
Reflection	Severity review step	Today
Tool Use	Query problem codes DB	Today + Week 3
Planning	LangGraph conditional routing	Week 3
Multi-Agent	Extraction → Classification → Routing	Later

Deep Dive: Reflection

The Idea

Instead of treating the first output as final, add a **feedback loop**:

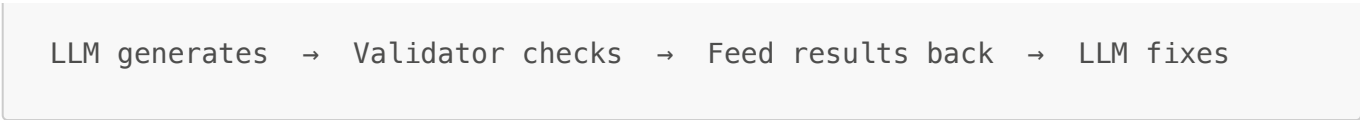
Generate draft → Review / critique → Revise → Better output

Two Flavors

Self-reflection:

LLM generates → LLM critiques → LLM improves

Reflection with external feedback:



External feedback is more powerful — grounded in **real data**, not just re-reading.

Does reflection actually work?

Self-Refine (Madaan et al., 2023) — tested across 7 tasks:

- Sentiment reversal, dialogue, code optimization, code readability, math reasoning, acronym generation, constrained generation

Result: Improved performance on **every task**.

Gains from **+3.6% to +49.2%** depending on the task.

Even strong models benefit from a second look.

Where Reflection Helps Most

Task	Problem Without Reflection	What Reflection Checks
Entity extraction	Misses compound hazards	"Safety risks underestimated?"
Structured data	Missing/malformed fields	Validate against schema
Summaries	Missing key details	"Includes problem, location, severity?"
Severity classification	Defaults to Medium	"Do urgency indicators match severity?"

CBRE Example

"Water's coming through the ceiling on the 4th floor, right above the server room. It started as a drip but it's getting worse. I'm concerned about the equipment."

Without Reflection

- Problem: water leak
- Severity: **Medium**

Misses: water + electrical equipment = safety hazard

With Reflection

1. First pass: water leak, severity Medium
2. Reflection: *"Are there safety risks being underestimated? Consider compound factors."*
3. Revised: severity → **High** or **Critical**

Catches the **interaction** between water and server room equipment.

Using Different Models for Reflection

LLM 1 — Generator (gpt-5-mini)

"Extract key entities from this transcript."

LLM 2 — Critic (gpt-4o)

"Critique the severity assessment. Check for compound hazards, understated urgency, life-safety risks."

Cheaper model generates. Stronger model reviews.

Reflection Prompt Tips

1. **Be explicit** — "reflect" / "review", not just "respond"
 2. **Give specific criteria** — what to check for
 3. **Feed in the original output** — can't improve what you can't see
 4. **Use external signals** — rule checks beat self-review
 5. **Try different models** — reasoning models critique well
-

Discussion

Where in the CBRE pipeline would reflection help the most?

Where would it be overkill?

Evaluations

The #1 Predictor of Success

Not technical skill.

Whether the team runs a **disciplined evaluation process**.

Measure → test → iterate → repeat.

Start Simple

1. **Look at outputs.** Read 20 results. Spot failure patterns.
 2. **Design a metric** for the most common failure.
 3. **Iterate.** Change the prompt, re-run, compare.
-

Objective Evals

Clear right/wrong answers:

Transcript	Predicted	True	Match?
TX-001	High	High	YES
TX-002	Medium	Critical	NO
TX-003	Low	Low	YES

This is what you'll do in today's notebook.

Subjective Evals

No single right answer? Use **binary rubric criteria**:

- Does the summary mention the problem type? *(yes/no)*
- Does it include the building name? *(yes/no)*
- Is it one sentence as requested? *(yes/no)*

LLM-as-Judge

Use a second LLM to evaluate outputs at scale.

Give it a rubric. Let it score.

Not perfect — but scales to hundreds of outputs.

The Loop

Evals reveal **what's broken** → Fix the prompt / reflection criteria / pipeline logic → Re-evaluate

Continuous improvement.

Framework Landscape

Framework	What It Does
LangChain	LLM abstraction — structured output, prompts, chaining
LangGraph	Workflow orchestration — graphs, state, routing
CrewAI	Multi-agent role collaboration
AutoGen	Multi-agent conversations (Microsoft)

We're using **LangChain + LangGraph**.

Today: LangChain. Week 3: LangGraph.

Bookmark: python.langchain.com — you'll use this a lot.

Recap

Concept	One-Liner
Agentic AI	Iterative, not one-shot
Autonomy	A spectrum — pick the right level
Benefits	Better performance, parallelism, modularity
Task Decomposition	LLM vs. tool vs. code for each step
Design Patterns	Reflection, tool use, planning, multi-agent
Reflection	Review loop — catches missed mistakes
Evaluations	Measure systematically

Demo Time

[entity_extraction_demo.ipynb](#)

Transcript → Structured Entity Extraction → Evaluation → Reflection

The first node of the pipeline we'll build through Week 10.

Assignment

Time estimate: ~3 hours total (including setup, learning, exploring)

Part 1: Think Like a Builder

~45 min — pen & paper or Google Doc

1a. Decompose the Pipeline

Write out the steps your CBRE call routing system would follow, from transcript arrival to routing decision.

For each step, label it:

- **LLM** — requires language understanding or generation
- **Tool** — database query, API call, or code execution
- **Code** — simple deterministic logic (if/else, thresholds)

Aim for at least 4-5 steps.

1b. Where Would You Add a Human?

Pick 1-2 points in your pipeline where a human should review before the decision goes through.

Explain **why** at that point — what could go wrong if the AI acted alone?

1c. Write a Tricky Transcript

Write one short maintenance call (~3-5 sentences) that would be **hard** for an AI to classify.

Ideas for trickiness:

- Ambiguous severity
 - Multiple problems at once
 - Calm caller + dangerous situation
 - Vague or slang-heavy language
-

Part 2: Hands-On Notebook

~1.5–2 hours

Open [entity_extraction_demo.ipynb](#) and work through it.

Part 2 Exercises (in the notebook)

1. **Run the notebook end-to-end** — note your baseline accuracy
 2. **Improve the prompt** — modify severity guidelines, re-run, compare accuracy
 3. **Add a new field** — extend the Pydantic model, check if the LLM fills it in sensibly
 4. **Test your tricky transcript** — paste the one you wrote in 1c, see what happens
-

What to Bring to Week 3

- Your pipeline decomposition from 1a (we'll compare)
 - Your baseline accuracy and whether prompt changes helped
 - Your tricky transcript and how the system handled it
-

Quiz: Check Your Understanding

Q1

You build a system that: (1) takes a maintenance transcript, (2) calls an LLM to extract entities, (3) uses those entities to query a building database via API, (4) calls the LLM again to draft a routing recommendation, (5) runs a second LLM pass to critique and revise the recommendation. A colleague says: "This isn't really agentic — you hard-coded steps 1-5." How do you respond?

- (A) They're right — truly agentic systems must choose their own steps, this is just a pipeline

- (B) It's agentic because it uses an LLM — any LLM-based system counts
 - (C) It's agentic at the low-autonomy end — it combines LLM reasoning, tool use, and a reflection loop, even though the steps are fixed
 - (D) It would only be agentic if you added a human-in-the-loop
-

Q2

Three CBRE systems: **A** always runs extract → classify → route (fixed). **B** extracts entities, then the LLM picks which of 5 tools to call based on what it found. **C** extracts entities, plans a multi-step investigation, writes a custom SQL query it's never seen before, executes it, then decides its next action based on the result. Rank them by autonomy.

- (A) A = low, B = low, C = semi — only C involves any real decision-making
 - (B) A = low, B = semi, C = highly autonomous
 - (C) A = not agentic, B = low, C = semi
 - (D) All three are semi-autonomous — they all use LLMs and tools
-

Q3

Your extraction system gets 85% severity accuracy. You try two improvements: (I) upgrade from GPT-5 mini to GPT-5 (same prompt, no other changes), which bumps accuracy to 89%. (II) Keep GPT-5 mini but add a reflection step where a second LLM call reviews the extraction, which bumps accuracy to 93%. A teammate says "Just use the bigger model — it's simpler." What's the strongest reason they might be wrong?

- (A) GPT-5 is too expensive to use in production
 - (B) The reflection approach gets better results AND demonstrates that architecture (adding a review loop) can outperform simply scaling the model — and you can combine both later
 - (C) GPT-5 mini is always better than GPT-5
 - (D) Reflection is simpler to implement than switching models
-

Q4

A caller says: "Hey, just a heads up — the sprinkler in the server room went off for about five seconds then stopped. Everything looks dry, no alarms going off. Figured I'd let you know." Your system extracts severity: Low. The reflection step needs to evaluate this. Which reasoning is correct?

- (A) Keep Low — the caller explicitly says everything looks dry, no alarms, and no rush. Reflection should respect the caller's assessment since they're the one on-site
 - (B) Upgrade to High — sprinkler activation in a server room implies a heat/fire trigger event, and any water exposure near electronics is a compound hazard, regardless of the caller's calm tone
 - (C) Upgrade to Medium — it's worth investigating but the fact that it stopped on its own and there's no visible damage means it's not high-urgency
 - (D) Upgrade to Critical — any sprinkler activation must be treated as a fire event
-

Q5

Your CBRE pipeline does this sequence: (1) LLM extracts entities from transcript. (2) LLM looks at the extraction and decides: "Severity is High and the building is in Zone A, so I should check the Zone A vendor contract first, then pull the building's last 3 work orders." (3) System calls the vendor API and work-order database. (4) LLM synthesizes everything into a routing recommendation. Which patterns appear in which steps?

- (A) Steps 1, 4 = tool use; Step 2 = planning; Step 3 = reflection
 - (B) Step 1 = tool use; Step 2 = planning; Step 3 = tool use; Step 4 = reflection
 - (C) Step 1 = tool use (structured output); Step 2 = planning; Step 3 = tool use (API calls); Step 4 = generation (no pattern — just LLM output)
 - (D) Step 1 = planning; Step 2 = reflection; Steps 3-4 = tool use
-

Q6

Your system extracts severity and a one-sentence summary. You build evals for both. For severity, you compare against human labels and get 87% accuracy. For summaries, you write a rubric with 4 criteria (mentions the problem, includes location, states severity, is one sentence) and have GPT-5 score each summary. A teammate argues: "The summary eval is unreliable because we're using an LLM to judge an LLM." What's the best response?

- (A) They're right — LLM-as-Judge is fundamentally unreliable and should never be used
 - (B) The rubric mitigates this — each criterion is binary and specific ("does it mention location: yes/no"), making the judge's task narrow and verifiable, unlike open-ended "rate the quality"
 - (C) The summary eval is actually objective too — just exact-match against reference summaries
 - (D) You should replace the LLM judge with a keyword-matching script for all four criteria
-

Q7

You test two reflection setups on the same 50 transcripts. **Setup A:** A single prompt that says "Review this extraction for errors and fix any issues with severity, location, or missing fields." **Setup B:** Three separate, focused prompts run sequentially — one checking severity against compound hazard rules, one verifying location completeness, one cross-referencing urgency indicators against the assigned severity. Setup B costs 3x more in API calls. When is Setup A the better choice?

- (A) Never — more specific prompts always produce better results
 - (B) When the extraction task is straightforward and errors are rare — the marginal improvement from Setup B isn't worth 3x the cost for transcripts where the first pass is usually correct
 - (C) When you need faster results — Setup A runs in one call vs. three
 - (D) Both B and C — cost and latency both matter, and for routine transcripts the single pass is often sufficient
-

Q8

Your reflection loop uses the same GPT-5 mini for both extraction and review. It catches 60% of severity errors. You try three alternatives: (I) Use GPT-5 (stronger model) as the reviewer — catches 72%. (II) Keep GPT-5 mini as reviewer but add a keyword scan that flags dangerous terms ("gas," "fire," "trapped") and

feeds the flags into the reflection prompt — catches 81%. (III) Combine both: GPT-5 reviewer + keyword flags — catches 83%. What does this tell you about reflection design?

- (A) Always use the strongest model available for reflection
- (B) External signals (keyword flags) provide more marginal value than a model upgrade because they surface evidence the model missed, not just a second opinion from a stronger reasoner
- (C) The keyword scan alone would be sufficient — the LLM reviewer adds little value
- (D) The combined approach (III) is always worth the extra cost

Q9

You're decomposing a step in the CBRE pipeline: "Determine whether this situation requires notifying the building's insurance provider." Which is the right approach, and why?

- (A) Deterministic code — check if `severity == Critical`, if so notify insurance
- (B) LLM — the decision depends on interpreting the problem description, not just severity (e.g., a Critical gas leak might need insurance, but a Critical elevator entrapment might not involve property damage)
- (C) Tool call — query the insurance provider's API to check coverage
- (D) This step should be further decomposed: LLM to assess if property damage is likely, then deterministic code to check the policy threshold, then a tool call to notify

Q10

You've built a modular CBRE pipeline: Extraction → Severity Check → Routing → Notification. Each module has its own eval suite. The extraction module scores 90% accuracy. The severity module scores 95% on correctly-extracted inputs. End-to-end, your system routes correctly ~82% of the time. A teammate says: "Each module is great — the 82% must be a measurement error." What's actually happening?

- (A) They're right — $90\% \times 95\% = 85.5\%$, so 82% is within normal variance
- (B) Errors compound through the pipeline — the extraction module's 10% errors become the severity module's garbage input, and each module's mistakes cascade, so end-to-end accuracy is lower than any individual module's score
- (C) The routing and notification modules must have bugs since extraction and severity are fine
- (D) The eval suites are measuring different things and can't be compared

Quiz Answers

Q	Answer	Why
1	C	Fixed steps don't disqualify a system from being agentic — low autonomy is still on the spectrum. The LLM reasoning + tool use + reflection loop make it agentic
2	B	A = developer-determined steps (low). B = LLM chooses among tools (semi). C = LLM creates novel actions and adapts (highly autonomous)

Q	Answer	Why
3	B	The point isn't that bigger models are bad — it's that architecture improvements (reflection) gave a bigger lift than model scaling, and you can stack both
4	B	Sprinkler activation = heat trigger (possible fire) + water in a server room = compound hazard. Calm tone $\neq$ low severity — reflection should catch what tone-based assessment misses
5	C	Step 1 uses the LLM like a tool (structured extraction). Step 2 is planning (deciding what to do next). Step 3 is tool use (executing API calls). Step 4 is LLM generation — not every LLM call is a "pattern"
6	B	LLM-as-Judge works when the rubric criteria are binary and specific. The narrower the judge's task, the more reliable it is. Open-ended "rate quality" is unreliable; "does it mention location: yes/no" is not
7	D	Both cost and latency matter. For straightforward transcripts where errors are rare, a single focused prompt is often sufficient — the 3x cost of Setup B only pays off on difficult or high-stakes cases
8	B	External signals add <i>new information</i> the model didn't have. A stronger model still only re-reads the same text. That's why (II) beats (I) despite using a weaker model — evidence > reasoning power
9	D	This is a multi-faceted decision that can't be reduced to one step. It needs LLM judgment (property damage likely?), code logic (policy threshold), and a tool call (notification API) — decompose further
10	B	Error cascading — each module's mistakes become the next module's bad inputs. $90\% \times 95\% \times \text{routing accuracy} \times \text{notification accuracy}$ compounds down. This is why end-to-end evals matter, not just per-module scores

Food for Thought

No right or wrong answers — just things worth thinking about.

FT-1

Your CBRE system gets severity right 90% of the time. The remaining 10% are edge cases — ambiguous transcripts where even two human dispatchers might disagree. **Should you keep trying to push accuracy higher, or invest that effort into building a better human-in-the-loop for the uncertain cases?** Where's the point of diminishing returns?

FT-2

Reflection improves results, but every reflection step is another LLM call — more cost, more latency. **When is reflection *not* worth it?** Is there a type of task where the first pass is "good enough" and the review step is wasted money?

FT-3

We said a weaker model with agentic patterns can outperform a stronger model used one-shot. But models keep getting better — GPT-5 is much stronger than GPT-3.5 was. **Will agentic architectures still matter in 2-3 years, or will models get good enough that a single pass is sufficient?** What would have to be true for agentic patterns to become obsolete?

FT-4

In the CBRE pipeline, we let the LLM decide severity. But severity determines whether 911 gets called. **How much autonomy should an AI system have when the stakes are life-safety?** Where would you draw the line between "the AI decides" and "a human must confirm"?

FT-5

Today we used one LLM to generate and the same LLM to reflect. **Is the model really "catching its own mistakes," or is it just getting a second chance with more context?** What's the difference between genuine self-reflection and just running inference twice? Does it matter if the result improves either way?

Next Week

Week 3: LangGraph & RAG

Connect the extraction step to a problem code database.

Build a real multi-step pipeline.
